Problem Discussion

Arman Ferdous

Ternary Search

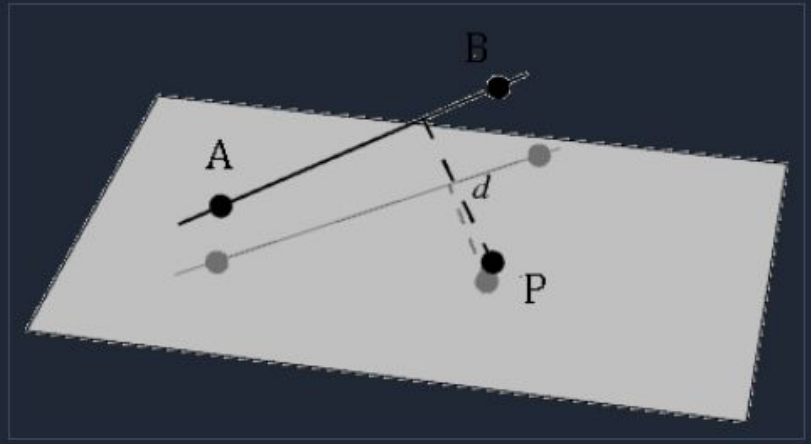
- Instead of checking with one midpoint, we evaluate the function at two thirds, and then make our decision.
- A good question to ask is what this gives us?
- Many people teach this wrong and say that this is an improved version of the binary search and both do the same thing. But it’s not like that.
- The ternary search is used to find the highest/lowest point of a concave/convex function.

A warning on discrete functions

- Sometimes the function might be discrete (e.g. defined only on integers).
- In that case, the function MUST satisfy unimodality
  - Unimodal: When the function is strictly increasing and then strictly decreasing (or vice versa). In other words, the discrete values must form a single peak or a single valley without multiple local extrema.
- The derivative (finite difference) trick: Instead of ternary searching over f(x), you can consider binary searching on f(x) - f(x - 1). The sign of this value only changes at the peak/valley.

https://lightoj.com/problem/point-segment-distance-3d

Given a segment in a 3D space, identified by **A(x_1, y_1, z_1)**, **B(x_2, y_2, z_2)** and another point **P(x, y, z)** your task is to find the minimum possible Euclidean distance between the point **P** and the segment **AB**.



Input

Input starts with an integer **T (≤ 10000)**, denoting the number of test cases.

Each case starts with a line containing nine integers **x_1, y_1, z_1, x_2, y_2, z_2, x, y, z**. The magnitude of any integer will not be greater than **100**.

Output

For each case, print the case number and the distance. Errors less than 10^{-6} will be ignored.

```
int main () {
    double x1, x2, x;
    cin >> x1 >> x2 >> x;

    double lo = 0, hi = 1;
    for (int it = 0; it < 60; ++it) {
        double mid1 = (lo + lo + hi) / 3.;
        double mid2 = (lo + hi + hi) / 3.;

        double x_at_mid1 = x1 + (x2 - x1) * mid1;
        double x_at_mid2 = x1 + (x2 - x1) * mid2;

        if (dist(x_at_mid1, x) <= dist(x_at_mid_2, x))
            hi = mid2;
        else lo = mid1;
    }
    int x_opt = x1 + (x2 - x1) * lo;
    // now dist(x_opt, x) is answer
}
```

Egg Dropping Puzzle

- You are currently on a 100 storied building and have TWO eggs with you.
- You can throw an egg from a floor and verify whether the egg breaks.
- Both eggs have the same strength. And if the egg doesn't break after falling, assume that it doesn't cause any internal damage (so it's not weakened in future).
- You want to find the highest floor, from where the egg doesn't break when thrown.
- Find a strategy to minimize the number of throws required. You can break both the eggs in the process if needed.

https://toph.co/p/not-today

Alice is traveling through an undirected connected graph with n nodes and m edges, where multiple edges may connect the same pair of nodes. She starts at node a and wants to reach node b, while Bob tries to prevent her from reaching node b for as long as possible.

Her journey unfolds over a sequence of days. Each day, the following happens in order:

1. Bob blocks exactly one edge of the graph. The block lasts for the current day only; the next day the edge is usable again.
2. Seeing which edge is blocked, Alice either traverses one unblocked edge to an adjacent node, or stays where she is.

Alice plays to reach b in as few days as possible, and Bob plays to delay her for as many days as possible. Assuming both play optimally, determine the number of days Alice needs to reach b, or report that Bob can prevent her from ever reaching node b.

Input

The first line contains a single integer t ($1 \le t \le 10^5$) — the number of test cases.

The first line of each test case contains four integers n, m, a, and b ($2 \le n \le 2 \cdot 10^5$, $n - 1 \le m \le 5 \cdot 10^5$, $1 \le a, b \le n$, $a \neq b$) — the number of nodes, the number of edges, the starting node, and the destination node.

Each of the next m lines contains two integers u and v ($1 \le u, v \le n$, $u \neq v$) — an undirected edge connecting nodes u and v. It is guaranteed that the graph is connected.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$, and the sum of m over all test cases does not exceed $5 \cdot 10^5$.

https://toph.co/p/the-paintress

You have n boards and n paint colors. The i-th board has length l_i, and there are c_j units of paint of color j.

The total length of all boards equals the total amount of paint: $\sum_{i=1}^{n} l_i = \sum_{j=1}^{n} c_j$.

You must use all the paint and fully cover every board. Formally, construct a matrix x of size $n \times n$, where $x_{i,j}$ is a non-negative integer equal to the number of units of board i painted with color j. It must satisfy $\sum_{j=1}^{n} x_{i,j} = l_i$ for every board i, and $\sum_{i=1}^{n} x_{i,j} = c_j$ for every color j.

A painting is called simple if the boards can be ordered as $p_1, p_2, \ldots, p_n$ so that, for every k from 1 to n, the first k boards $p_1, p_2, \ldots, p_k$ use at most k distinct colors in total — that is, the number of colors appearing on at least one of these k boards is at most k.

It can be shown that a simple painting always exists. You must output a simple painting together with such an order as proof that it is simple.

Input

The first line contains a single integer t ($1 \le t \le 100$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 1000$) — the number of boards and colors.

The second line contains n integers $l_1, l_2, \ldots, l_n$ ($1 \le l_i \le 10^9$) — the lengths of the boards.

The third line contains n integers $c_1, c_2, \ldots, c_n$ ($1 \le c_j \le 10^9$) — the amounts of each color of paint.

It is guaranteed that $\sum_{i=1}^{n} l_i = \sum_{j=1}^{n} c_j$, and that the sum of n over all test cases does not exceed 1000.

https://www.facebook.com/codingcompetitions/hacker-cup/2025/practice-round/problems/B

When you use the Meta Quest, you set up a "Guardian boundary", a safe zone for you to move around in within VR. You're trying to figure out the size of the largest Guardian boundary that you can make inside your house.

Your house is represented as an $R \times C$ grid surrounded by a wall on each side, where every cell is either `.`, representing empty space, or `#`, representing an object.

To ensure that you don't hit anything while playing, you want to find the largest contiguous space in your house such that every cell is at least S spaces between itself and the nearest object or wall. **Formally, the shortest path (traveling only up, down, left, or right) from a safe zone cell to the nearest object or wall must visit S or more spaces, not including the start/end cells.**

Constraints

$1 \leq T \leq 70$
$1 \leq R, C \leq 1{,}000{,}000$
$1 \leq R \times C \leq 1{,}000{,}000$
$1 \leq S \leq 1{,}000$

The sum of $R \times C$ across all test cases is at most $5{,}000{,}000$.

Input Format

Input begins with an integer T, the number of test cases. Each case starts with a line that contains the integers R, C, and S. Then R lines of C characters follow, representing your house. Every character is either `.` or `#`.

Output Format

For the ith test case, print "`Case #i: `" followed by the size of the largest contiguous space you can play in that satisfies the safety parameter, S.

```
5
5 5 1
.....
.....
.....
.....
.....
7 7 2
.......
.......
..#.#..
..#....
.......
.......
.......
7 7 1
.......
.......
..#.#..
..#....
.......
.......
.......
```

```
Case #1: 9
Case #2: 0
Case #3: 10
```

https://codeforces.com/contest/2208/problem/D1

This is the easy version of the problem. The difference between the versions is that in this version, the constraint on n is lower. You can hack only if you solved all versions of this problem.

You once had an undirected tree with n nodes. To make the tree look more interesting, you decided to assign an arbitary direction to each of the $n - 1$ edges.

As time goes by, you forgot the structure of your tree. However, you found a note which recorded **after** the direction of the edges have been assigned, whether u can reach v* for all ordered pairs of (u, v) which satisfies $1 \leq u, v \leq n$.

You want to find out the structure of the tree and the direction of the edges from the information given by the note. Determine if there is possible solution and construct one. If there are multiple solutions, you only need to find one of them.

---

*For a directed graph, we say that x can reach y if and only if there exists a sequence of nodes $u_1, u_2, \ldots, u_k$ such that $u_1 = x, u_k = y$ and for all i from 2 to k, the directed edge $u_{i-1} \to u_i$ exists. In particular, a node can always reach itself.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \leq t \leq 10^4$). The description of the test cases follows.

The first line of each test cases contain an integer n ($2 \leq n \leq 500$) denoting the number of nodes your tree have.

The following n lines contain a string s_i. s_i is of length n and consists only of 0 and 1. The j-th character of s_i is 1 if and only if i can reach j after the edges are directed.

It is guaranteed that the sum of n^3 over all test cases does not exceed 500^3.

https://toph.co/c/inter-university-sust-cse-carnival-2024

You are given $2n$ distinct points on a two-dimensional plane. You must partition these $2n$ points into exactly n pairs (that is, every point must belong to exactly one pair) and connect each pair of points with a straight line segment, producing n line segments in total.

Find a pairing such that all of the n segments are parallel to one another and no two of them intersect, meaning no two segments share a common point. If such a pairing exists, print any one of them; otherwise, report that it is impossible.

Input

The first line contains a single integer t ($1 \leq t \leq 100$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 1000$) — the number of pairs.

Each of the next $2n$ lines contains two integers x_i and y_i ($-10^9 \leq x_i, y_i \leq 10^9$) — the coordinates of the i-th point.

It is guaranteed that all $2n$ points in each test case are distinct and that the sum of n over all test cases does not exceed 1000.

Output

For each test case, if no valid pairing exists, print -1.

Otherwise, print n lines. Each line should contain two integers u_i and v_i ($1 \leq u_i, v_i \leq 2n$, $u_i \neq v_i$) — denoting that the u_i-th and v_i-th points form a pair.

If there are multiple valid pairings, output any of them.

https://toph.co/c/inter-university-sust-cse-carnival-2024

You are given a string s consisting of lowercase English letters.

Find two non-empty substrings* p and q of s such that p is not a substring of q, and q is not a substring of p. Note that p and q do not have to be disjoint — they may correspond to overlapping positions in s. Over all such pairs (p, q), output the maximum possible value of $|p| + |q|$, where $|x|$ denotes the length of a string x.

If no such pair exists, output 0.

---

*A string a is a substring of a string b if a can be obtained from b by deletion of several (possibly, zero or all) characters from the beginning and several (possibly, zero or all) characters from the end. For example, `us` is a substring of `sust`, but `ut` and `aus` are not.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first and only line of each test case contains a string s ($1 \le |s| \le 3 \cdot 10^5$) consisting of lowercase English letters.

It is guaranteed that the sum of $|s|$ over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, output a single integer — the maximum value of $|p| + |q|$ over all valid pairs, or 0 if no valid pair exists.

https://codeforces.com/contest/2208/problem/D1

The difference between the easy version and hard version is the maximum number of queries allowed. In this version, it is 33.

There is a secret array a of length $2n + 1$, whose elements are integers from 1 to n. Each value appears exactly twice, except for one value, which appears exactly three times.

Your goal is to find the three positions of the value that appears three times.

To do this, you may ask the following query at most 33 times:

1. Choose an integer k and an array s of k distinct indices between 1 and $2n + 1$.
2. You will receive the number of different values among $a_{s_1}, a_{s_2}, \ldots, a_{s_k}$ that appear exactly once, or, in other words, the count of values that are not repeated.

For example, if the values $a_{s_1}, \ldots, a_{s_k}$ are $\{2, 1, 2, 3, 2, 3, 6, 7\}$, the answer to the query will be 3, as 1, 6, 7 are the only values that appear exactly once. 3 appears 2 times and 2 appears 3 times, which is more than once, so they are not counted.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 500$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \le n \le 1000$).

The array a is fixed for the test case and does not change during the interaction. In other words, the interactor is **not adaptive**.

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^4$.

Solution Links

Not today

The Paintress

Tree Orientation

Parallel Lives, Standoff

Unique Values